CSE 213 — Software Engineering

Use Case Diagrams and BPMN Process Models

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Anindya Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — use-case and UML-overview questions; BPMN = insurance
Lesson 19 — UML Overview, Actors, Use Cases and Their Relationships — Use case.pptx sl. 1–16
Lesson 20 — Worked Use Case Diagrams and Use Case Narratives — TF scenarios: Mentcare, bank, hotel, rent-a-car …
Lesson 21 — BPMN Core: Events, Activities, Gateways — 3 - BPMN.pptx sl. 1–20
Lesson 22 — BPMN Pools, Lanes, Message Flows and Artifacts — 3 - BPMN.pptx sl. 21–34
Quick Notes — Use Cases and BPMN — revision + key diagrams

> **NOTE** Slide numbers refer to the PDF page numbers of the deck named under each lesson title. TF page numbers refer to 'CSE 213 All Past Papers.pdf' (70 pages); the newest paper (22 batch, labelled 2024-25 here) is a separate file, cited as '22 batch p.N'. Old course code: CSE 307. Lessons 19–20: 'Use case.pptx' (16 slides). Lessons 21–22: '3 - BPMN.pptx' (34 slides). Figures are taken from PDF copies of the pptx decks made with PowerPoint; slide numbers are the pptx slide numbers. Diagrams in solutions are written as text: actor —— (use case), (A) ··«include»··> (B), and for BPMN ○ start, ◎ end, [task], <X> XOR, <+> AND, <O> OR.

Exam Map — What the TF Papers Ask

Session (paper)	Questions in this note file
2023-24 (CSE 213)	Q1(a) Mentcare: use case diagram of the processes through which a criminal can access the system (7) [L20]
2016-17 (CSE 307)	Q1(a) Extensibility mechanisms of UML (12) [L19] · Q2(a) Bank (normal / foreign users) — use case diagram (13) [L20] · Q2(b) Use case narrative for 'Withdraw money' (7⅔) [L20]
2014-15 (CSE 307)	Q5(c) UML diagram types; static vs dynamic (10) [L19] · Q6(d) Book / author / publisher / bookstore / reader — use case diagram (9⅔) [L20]
2013-14 (CSE 307)	Q1(c) <<extend>> and <<include>> (uses) relationships with examples (7⅔) [L19] · Q1(d) Bank teller, exam-fee deposit queue — extended use case (15) [L20] · Q5(b) Four kinds of things in UML (12) [L19] · Q6(d) Online bus reservation — use case diagram (22) [L20]
2012-13 (CSE 307)	Q7(d) Kinds of things in UML (4⅔) [L19] · Q8(a) Northeast Hotel reservations — use case diagram + use case dependency diagram (20) [L20]
2011-12 (CSE 307)	Q2(a) ABC Car Rentals: use case narrative 'Rent a Car' (18⅔) [L20]
2010-11 (CSE 307)	Q1(b) Behavioural UML diagrams and their purpose (8) [L19] · Q2(c) Book / author / publisher / store / reader — use case diagram (8) [L20]

EXAM FOCUS Use case diagrams: 2023-24 Q1(a) (Mentcare, ★★★) and five older papers; use case narratives: 2016-17, 2013-14, 2011-12. UML overview questions (diagram types, kinds of things, extensibility) are 5–12 marks in older papers. BPMN has not been asked yet — it is new in this syllabus, so treat Lessons 21–22 as insurance and learn the gateway rules (a 10-mark 'model this process in BPMN' is the likely form).

Lesson 19 — UML Overview, Actors, Use Cases and Their Relationships

Slides 1–16 (Use case.pptx)

1. UML (sl. 1–2)

UML = Unified Modeling Language: an international industry-standard graphical notation for describing software analysis and design — pictures (views) of an OO system. Diagrams listed: use case, class, sequence, collaboration, statechart, object, activity, component, deployment.

Static (structure)	Dynamic (behaviour)
Class, Object, Component, Deployment, Package, Composite structure	Use case, Activity, State machine (statechart), Sequence, Communication (collaboration), Timing, Interaction overview

> NOTE Textbook background for older questions. **Four kinds of things in UML:** structural things (class, interface, use case, component, node), behavioural things (interaction, state machine), grouping things (package), annotational things (note). **Extensibility mechanisms:** stereotypes («entity», «include»), tagged values ({version = 2.1}) and constraints ({ordered}, {balance ≥ 0}). **Behavioural diagrams:** use case (functions from the user's view), activity (workflow), state machine (life of one object), and the interaction diagrams sequence / communication / timing / interaction overview (message exchange).

2. Use cases, actors, stakeholders (sl. 3–7)

- A use case is a sequence of actions a system performs that yields a valuable result for a particular actor — a scenario-based description of how the system interacts with its environment.
- **Actor:** anything with behaviour outside the system that acts on it — a human role, a hardware device, an external system, or time. Drawn as a stick figure with its name. A single actor may represent many people.
- **Primary actor** initiates the interaction to achieve a goal; **supporting actor** performs sub-goals; a **stakeholder** is anyone interested in the system and may never act in a scenario.
- Finding actors (sl. 5): who is interested, where is it used, who benefits, who supplies/uses/removes information, does one person play several roles (one actor per role), do several people play the same role (one actor).

3. Diagram elements and relationships (sl. 8–15)

Element	Notation	Rule / example
Use case	ellipse with a verb–noun name	one function, may contain several scenarios (paths): Make appointment
Association	plain line (or double-headed arrow) actor — use case	communication; Patient — Make appointment
System boundary	rectangle around the use cases, actors outside	defines scope; may be one module of an ERP
Generalisation	line with hollow triangle from child to parent	New Patient, Old Patient → Patient; Make new/old patient appt → Make appointment
«extend»	dashed arrow **from the extension to the base**	optional behaviour: Update patient information ⋯> Make appointment
«include»	dashed arrow **from the base to the included** use case	mandatory reused behaviour: Manage schedule ⋯> Record availability

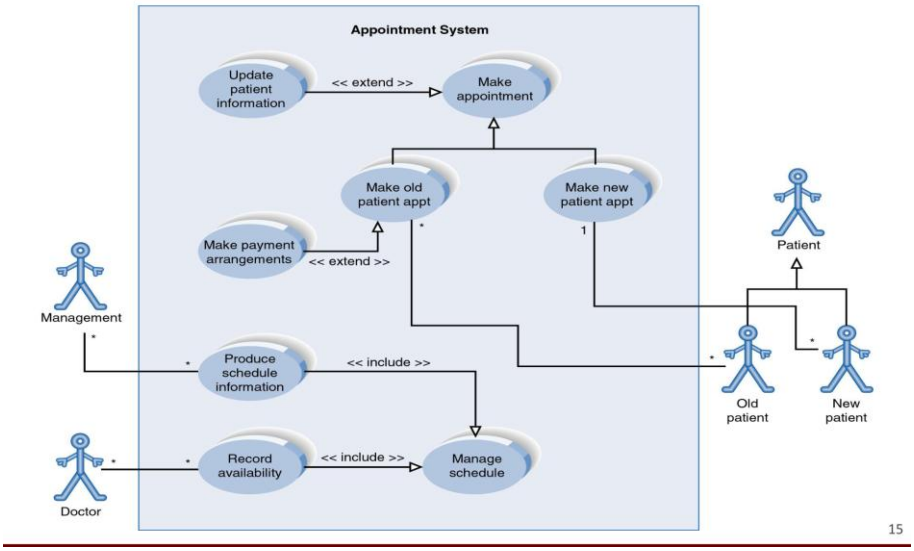


Figure 19.1 — Appointment system with generalisation, include and extend (sl. 15)

TRAP Directions: «include» points to the use case that is always executed as part of the base; «extend» points to the base it may optionally extend. (Slide 14's picture draws the include arrow from Record availability to Manage schedule; slide 14's text and standard UML say base → included. Follow the text and say so.) Old papers call «include» 'uses'.

Practice — Lesson 19

(a) Actual — 2013-14 Q1(c). (b) Actual — 2014-15 Q5(c). (c) Actual — 2013-14 Q5(b) / 2012-13 Q7(d). (d) Actual — 2016-17 Q1(a). (e) Actual — 2010-11 Q1(b).

(a) Define extends and uses relationships in use case diagrams with real-life examples. (7⅔)

(b) What types of UML diagrams exist? Name each, give its purpose, classify as static or dynamic. (10)

(c) What are the (four) kinds of things in UML? (12 / 4⅔)

(d) Discuss some extensibility mechanisms of UML. (12)

(e) List all behavioral UML diagrams and describe the purpose of each. (8)

✓ Solution

(a) «extend» (extends): optional or conditional behaviour inserted into a base use case at an extension point; the base is complete without it; arrow from extension to base. Example (ATM): (Print receipt) ··«extend»··> (Withdraw cash) — only if the customer asks. «include» (uses): behaviour common to several use cases factored out and always executed; arrow from base to included. Example: (Withdraw cash) ··«include»··> (Authenticate user) and (Transfer funds) ··«include»··> (Authenticate user).

(b) Table in §1 plus one-line purpose each: class — static structure; object — snapshot of instances; component — physical modules; deployment — hardware nodes and artifacts; package — grouping; use case — functionality from the users' view; activity — workflow/business process; state machine — life cycle of one object; sequence — time-ordered messages; communication — messages over links; timing — state changes over time.

(c) Structural, behavioural, grouping, annotational things (NOTE above), each with two examples.

(d) Stereotypes — new model element kinds derived from existing ones, shown «name» or with an icon («interface», «entity», «include»); tagged values — name=value properties attached to elements ({author = Rahim}, {version = 1.2}); constraints — conditions in braces, possibly in OCL ({ordered}, {ward.gender = patient.gender}). Profiles bundle them for a domain.

(e) Use case, activity, state machine, sequence, communication (collaboration), timing, interaction overview — purpose of each as in (b).

Lesson 20 — Worked Use Case Diagrams and Use Case Narratives

TF scenarios (method from Use case.pptx sl. 3–15)

1. Use case narrative (textbook format — needed for 2016-17, 2013-14, 2011-12)

Section	Content
Use case name / ID	verb–noun name
Primary actor, stakeholders	who initiates; who cares
Preconditions	must be true before it starts
Trigger	event that starts it
Main success scenario	numbered actor / system steps
Extensions (alternate flows)	2a, 4a … with the step they branch from
Postconditions (success guarantee)	true after success
Special requirements, frequency, open issues	non-functional notes

Practice — Lesson 20

(a) Actual — 2023-24 Q1(a). (b) Actual — 2016-17 Q2(a)–(b). (c) Actual — 2011-12 Q2(a). (d) Actual — 2012-13 Q8(a). (e) Actual — 2013-14 Q6(d). (f) Actual — 2014-15 Q6(d) / 2010-11 Q2(c). (g) Actual — 2013-14 Q1(d).

(a) Mentcare: clinic staff log on with username and password (≥ 8 letters, no other checks); a receptionist may transfer data to the national PRS. A criminal poses as a relative, learns nurses' names from badges and family details, and guesses passwords. Draw a use case diagram showing the processes through which the criminal can gain access. (7)

(b) Bank with Normal and Foreign users (create account, deposit, withdraw with limits, update balance, bonus, interest, convert currency): (i) use case diagram (13) (ii) narrative for 'Withdraw money' (7⅔).

(c) ABC Car Rentals: write the narrative for 'Rent a Car'. (18⅔)

(d) Northeast Hotel: reservation (registered or new customer), unique reservation code, amend/cancel, check-in by receptionist, check-out with card payment validation, optional printed invoice, room types, monthly reports for the manager. (i) System use case diagram (ii) use case dependency diagram. (15+5)

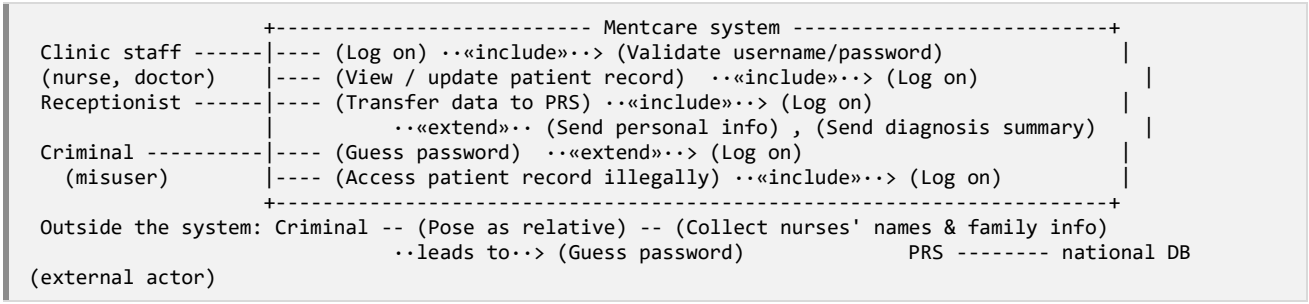
(e) Online bus reservation (buses, trips normal/express, seats, pay online or not, cancellation, authentication of customers and staff). Use case diagram. (22)

(f) A book is written by an author, published by a publisher, sold by a book store and read by a reader, who must buy it from a store selling it. Use case diagram with relationships. (9⅔ / 8)

(g) A bank teller serves students depositing exam fees in a queue: show the extended version of the use case. (15)

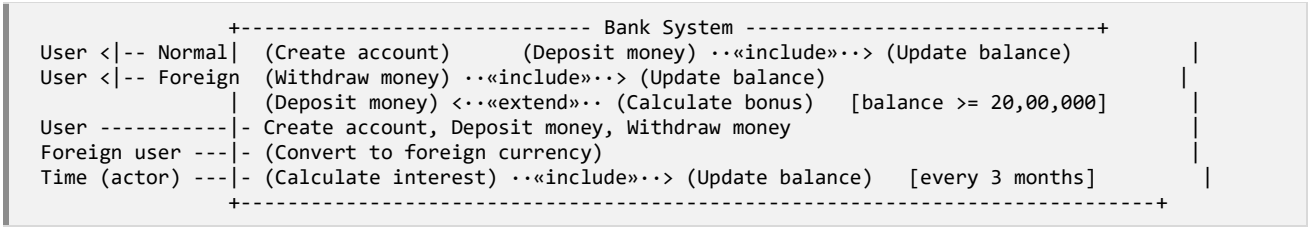
✓ Solution

(a) Misuse-case style: the criminal is an extra (hostile) actor; the processes he exploits are the normal log-on path plus the social-engineering steps outside the system.



Engagement points: (1) collecting names (badges, talking to nurses) — social engineering; (2) Log on with guessed passwords — weak policy (only length ≥ 8, no lockout); (3) once logged on, View patient record and Transfer data to PRS expose the star's diagnosis.

(b)(i)



(b)(ii) Narrative — Withdraw money

Field	Content
Primary actor	User (normal or foreign)
Precondition	User has an account and is authenticated
Main flow	1 User requests withdrawal and enters amount. 2 System checks amount ≤ 1,00,000. 3 System checks today's total + amount ≤ 3,00,000 and withdrawals today < 5. 4 System checks balance ≥ amount. 5 System debits the account (Update balance), increments today's count and total. 6 System dispenses money and shows the new balance.
Alternate flows	2a amount > 1,00,000: reject, ask for a smaller amount. 3a daily amount or count exceeded: reject, end. 4a insufficient balance: reject, end.
Postcondition	Balance reduced by amount; daily counters updated; transaction logged

(c) Narrative — Rent a Car

Field	Content
Primary actor / others	Customer (at the office); supporting actor: Employee (operates the system)
Precondition	Customer is at the office; employee logged in
Main flow	1 Customer gives time and duration. 2 Employee checks the system; system shows available car models. 3 Customer chooses a model. 4 System confirms availability for that time. 5 Employee asks for national ID and other information and enters it. 6 System books the specific car for the period. 7 Employee states rental cost and asks for 50 % advance. 8 Customer pays; system records payment and confirms the booking.
Alternate flows	4a model not available: system reports it; continue at step 3. 8a customer cannot pay the advance: booking cancelled, car released; use case ends.
Postcondition	Car booked for the period, customer recorded, advance received

(d)

```
                +---------------------------- Hotel Reservation System ----------------------------+
 Guest ----------|-- (Make reservation) ··«include»··> (Generate reservation code)                   |
                 |      <··«extend»·· (Register new customer)   [not registered]                     |
 Guest ----------|-- (Amend reservation)  ··«include»··> (Find reservation by code)                  |
 Guest ----------|-- (Cancel reservation) ··«include»··> (Find reservation by code)                  |
 Receptionist ---|-- (Check in)  ··«include»··> (Find reservation by code)                           |
 Receptionist ---|-- (Check out) ··«include»··> (Validate payment) ------------- Card Payment System|
                 |      <··«extend»·· (Print invoice)   [guest requests]                             |
 Hotel Manager --|-- (View monthly report)        Time ---- (Prepare monthly report)                |
                 +-------------------------------------------------------------------------------+
Use case DEPENDENCY diagram (order in which use cases must happen):
 Register customer --> Make reservation --> {Amend reservation | Cancel reservation} --> Check in --> Check
out
 Check out --> Prepare monthly report --> View monthly report
```

(e)

```
                +------------------------------ Bus Reservation System ------------------------------+
 Customer -------|-- (Search trips)   (Reserve seat) ··«include»··> (Authenticate)                     |
                 |   (Reserve seat) <··«extend»·· (Pay online)    [then seat cannot be cancelled]      |
                 |   (Cancel unpaid reservation) ··«include»··> (Authenticate)                        |
 Staff ----------|-- (Manage buses)  (Manage schedules & trips)  (Assign driver and hostess)          |
                 |   (Cancel no-show seats) [unpaid, 1 hour before trip] ··«include»··> (Authenticate) |
                 |   (Sell freed seat)                                                                |
 Trip <|-- Normal trip, Express trip  (generalised use cases: Plan normal trip / Plan express trip)    |
 Time -----------|-- (Release unpaid seats 1 h before departure)                                      |
 Payment gateway --- (Pay online)                                                                   |
                 +-------------------------------------------------------------------------------+
```

(f) System boundary = book trade; four actors.

```
 Author ---- (Write book)          Publisher ---- (Publish book) ---- Author
 Book store ---- (Sell book) ---- Reader          Reader ---- (Read book)
 (Read book) ··«include»··> (Buy book)    (Buy book) ··«include»··> (Sell book)   [only from a store
selling it]
 (Sell book) ··«include»··> (Publish book)        (Publish book) ··«include»··> (Write book)
```

(g) Extended use case = base use case plus its «include»/«extend» refinements.

```
 Student ---- (Deposit exam fee) ---- Teller
 (Deposit exam fee) ··«include»··> (Join queue / take token)
 (Deposit exam fee) ··«include»··> (Verify student ID and fee amount)
 (Deposit exam fee) ··«include»··> (Receive cash & issue receipt)
 (Pay late fine) ··«extend»··> (Deposit exam fee)        [after deadline]
 (Reject deposit) ··«extend»··> (Deposit exam fee)       [ID invalid / wrong amount]
```

Lesson 21 — BPMN Core: Events, Activities, Gateways

Slides 1–20 (3 - BPMN.pptx)

1. Core elements (sl. 1, 8)

Element	Symbol	Meaning
Activity (task)	rounded rectangle	work done: verb + noun ('Check stock availability')
Start event	thin circle ○	trigger; creates a token (token source)
End event	thick circle ◎	outcome; consumes the token (token sink)
Gateway	diamond ◇	split or join of the flow
Sequence flow	solid arrow	order of activities

2. Worked example — order-to-cash (sl. 2–7, 13)

Narrative: triggered by receipt of a purchase order; check stock; confirm or reject; if confirmed, emit invoice and ship goods; finish by archiving the order (or when rejected).

```
○ Purchase order received -> [Check stock availability] -> <X>
     <X> [items not in stock] -> [Reject order] -> ◎ Order rejected
     <X> [items in stock]     -> [Confirm order] -> <+> -> [Emit invoice] ---> <+> -> [Archive order]
-> ◎ Order fulfilled
                                                  <+> -> [Ship goods]  ---> <+>
```

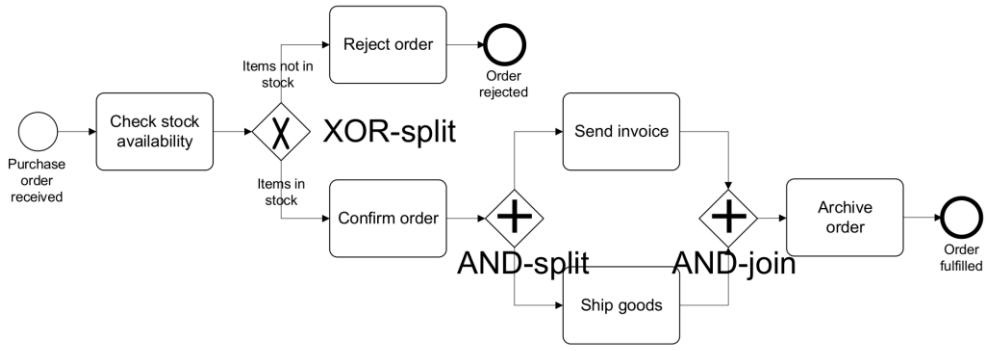


Figure 21.1 — Revised order-to-cash model with XOR and AND gateways (sl. 13)

3. Gateways (sl. 9–18)

Gateway	Split	Join	Example
XOR ◇ with ×	takes exactly one branch (labelled conditions; optional default flow)	proceeds when one incoming branch completes	invoice checking: post / re-send / block (sl. 10)
AND ◇ with +	takes all branches (parallel)	waits for all incoming branches	airport: security screening ‖ luggage screening (sl. 12)
OR ◇ with ○	takes one or more branches by condition	waits for all **active** branches	order distribution: Amsterdam and/or Hamburg (sl. 17–18)

Order distribution without OR (sl. 14–16): Solution 1 = XOR with three branches (only Amsterdam, only Hamburg, both → AND split/join); Solution 2 = AND split with an XOR 'skip' on each branch. The OR gateway (sl. 18) expresses it in one split/join pair.

TRAP Beginner's mistake (sl. 19): pairing an XOR split with an AND join deadlocks — the AND join waits forever for the branch that was not taken. Also, two activities on parallel branches need not run at the same time; they are only independent.

4. Naming guidelines (sl. 20)

1. Name every event and task.
2. Task = verb + business object (+ complement): Issue driver licence.
3. Message event = object + past participle: Invoice received, Claim settled.
4. Avoid generic verbs: Handle, Record …
5. Label every XOR-split branch with a condition: Policy is invalid.

Lesson 22 — BPMN Pools, Lanes, Message Flows and Artifacts

Slides 21–34 (3 - BPMN.pptx)

- **Pool** (sl. 21): a resource class — usually a business party (a whole company). **Lane**: a sub-class inside a pool — department (Sales, Warehouse), role (Manager), or software system (ERP). Lanes can be nested.
- **Message flow** (sl. 23): dashed line with open arrowhead between pools; to the pool boundary = informative message, to an activity/event = message that triggers it.
- **Syntactic rules** (sl. 25): sequence flow cannot cross pool boundaries (message flow can); both may cross lanes; a message flow cannot connect two elements in the same pool.
- **Data object** (sl. 28): artifact required or produced by an activity, with state in brackets (Purchase order [approved]). **Data store**: persistent storage (Warehouse DB, Orders DB). **Text annotation** (sl. 29): open bracket joined by a dotted line; does not affect token flow.

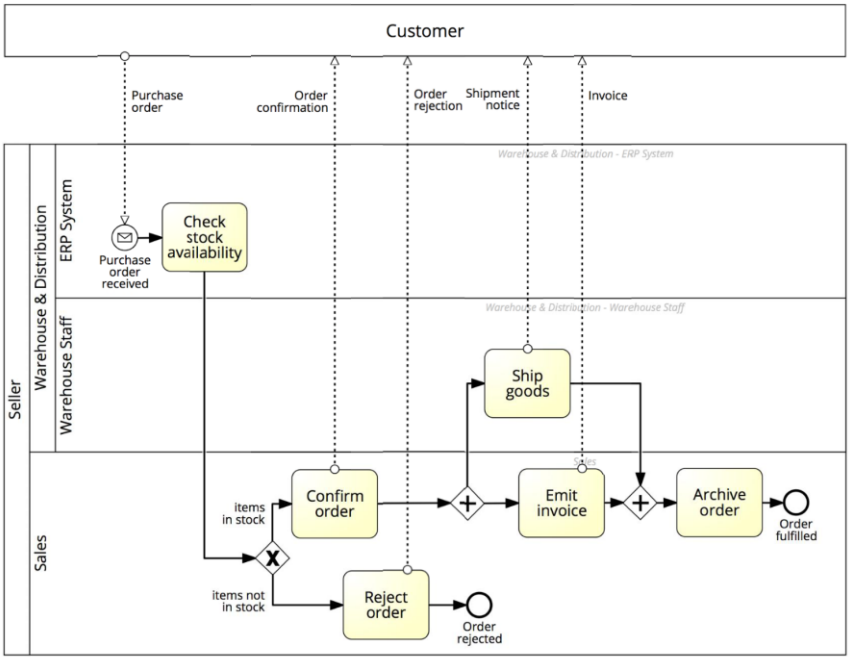


Figure 22.1 — Order-to-cash with lanes and a black-box Customer pool (sl. 24)

Practice — Lesson 22

(a) Constructed (likely TF form). (b) Constructed.

(a) Model in BPMN: a student submits an online course-drop request; the advisor reviews it; if rejected the student is notified; if approved, the accounts office computes the refund while the registrar updates the record, then the student is notified. Use pools/lanes and gateways.

(b) Why must an XOR-split not be closed by an AND-join? Give the fix.

✓ Solution

```
Pool Student (black box)  ··drop request··> Pool University
Pool University, lanes: Advisor | Accounts | Registrar | System
 Advisor:   ○ Drop request received -> [Review drop request] -> <X>
            <X> [request rejected] -> [Notify rejection] -> ◎ Request rejected   ··rejection··> Student
pool
            <X> [request approved] -> <+>
 Accounts:  <+> -> [Compute refund] ------------> <+>
 Registrar: <+> -> [Update academic record] ----> <+>
 System:    <+> (join) -> [Notify student of drop] -> ◎ Course dropped          ··confirmation··> Student
pool
 Data: Drop request [submitted] -> Review ; Student record (data store) <- Update academic record
```

(b) The XOR split passes the token to only one branch; an AND join waits for a token on every incoming branch, so it never fires — deadlock. Close an XOR split with an XOR join (or use AND split + AND join if both branches must run).

Quick Notes — Use Cases and BPMN

Revision points for Lessons 19–22.

L19 · Use case basics

- Actor = role outside the system (human, device, system, time); stakeholder may never act.
- Association line; boundary box; generalisation ▷; «include» base → included (always); «extend» extension → base (optional).
- UML things: structural, behavioural, grouping, annotational; extensibility: stereotypes, tagged values, constraints.

L20 · Worked

- Narrative: actor, precondition, main flow, alternate flows (2a…), postcondition.
- Mentcare: criminal as misuser actor; Guess password extends Log on.

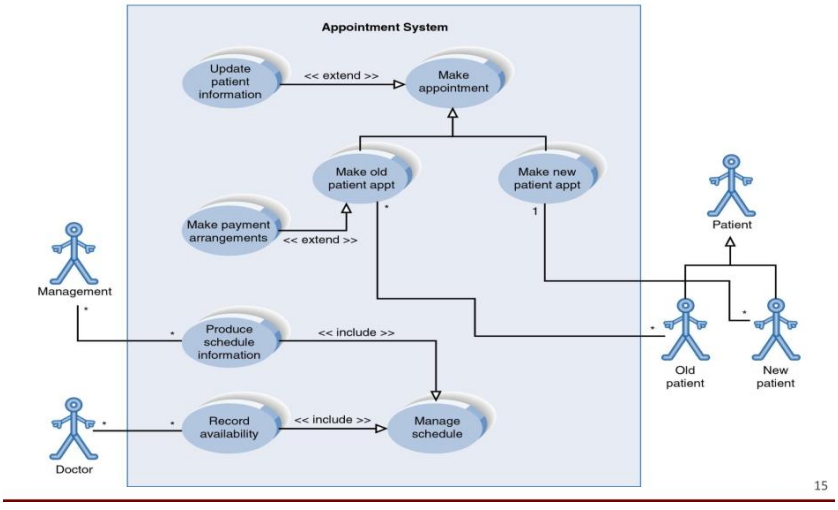
L21 · BPMN core

- ○ start, ◎ end, rounded task, ◇ gateway, arrow sequence flow.
- XOR = one branch; AND = all; OR = some, join waits for active ones. XOR split + AND join = deadlock.

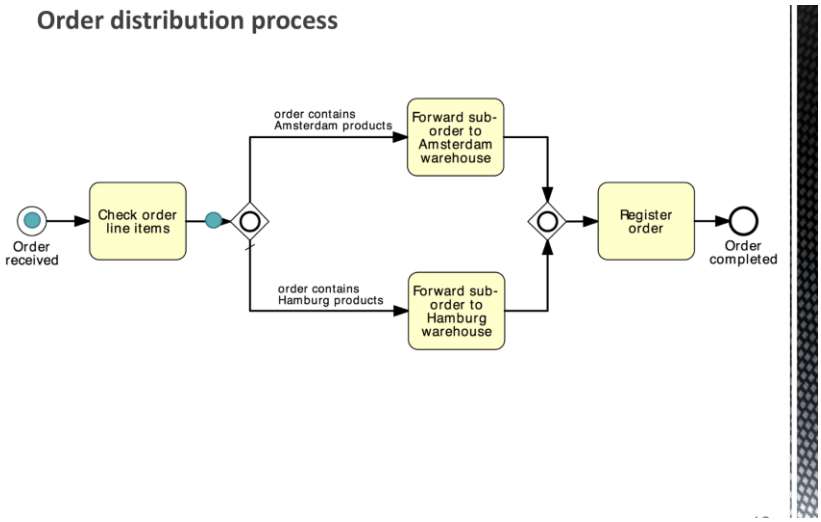
L22 · Pools

- Pool = party; lane = department/role/system; message flow only between pools.
- Data object [state], data store, annotation.

Key Diagrams to Reproduce



Use case diagram with relationships (sl. 15)



OR gateway solution (sl. 18)